

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 1

INFORME DE LABORATORIO

INFORMACIÓN BÁSICA					
ASIGNATURA:	COMPUTACIÓN GRÁFICA, VISIÓN COMPUTACIONAL Y MULTIMEDIA (E)				
TÍTULO DE LA PRÁCTICA:	<i>Procesamiento de Imágenes</i>				
NÚMERO DE PRÁCTICA:	<i>07</i>	AÑO LECTIVO:	<i>2026A</i>	NRO. SEMESTRE:	<i>IX</i>
FECHA DE PRESENTACIÓN	<i>09/06/26</i>	HORA DE PRESENTACIÓN	<i>13:00</i>		
INTEGRANTE (s): Venero Guevara Christian Henry				NOTA:	
DOCENTE(s): <i>MBA Mg. Ing. RENE ALONSO NIETO VALENCIA.</i>					

SOLUCIÓN Y RESULTADOS
I. SOLUCIÓN DE EJERCICIOS/PROBLEMAS 1. Introducción Esta práctica de laboratorio tiene como objetivo aplicar técnicas fundamentales de procesamiento digital de imágenes utilizando la biblioteca OpenCV en Python. Se implementan operaciones como redimensionamiento de imágenes, combinación de canales de color, conversión a negativo y escala de grises, umbral binario, dibujo de figuras y texto, y un programa interactivo de dibujo con eventos de mouse y teclado. Cada ejercicio permite comprender cómo las imágenes digitales se representan como arreglos de píxeles y cómo se pueden manipular para obtener resultados específicos, sentando las bases para aplicaciones más complejas en visión computacional y multimedia. 2. Redimensionar imágenes 2.1. Importación de la biblioteca OpenCV <pre>Python import cv2</pre> Esta línea importa la biblioteca OpenCV (Open Source Computer Vision Library). OpenCV es una biblioteca especializada en visión por computador y procesamiento de imágenes. Al importarla, tenemos acceso a todas sus funciones como leer, mostrar, redimensionar y guardar imágenes.

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 2

2.2. Carga de las imágenes desde el disco

Python

```
img1 = cv2.imread("Imagenes/Aguila.jpg")
img2 = cv2.imread("Imagenes/Gato.jpg")
img3 = cv2.imread("Imagenes/Lobo.jpg")
```

La función `cv2.imread()` lee una imagen almacenada en el disco y la carga en memoria como un arreglo multidimensional (matriz). Cada imagen se almacena en una variable diferente (`img1`, `img2`, `img3`). En este caso, las imágenes están en una carpeta llamada "Imagenes". La función devuelve un arreglo de números que representa cada píxel de la imagen.

2.3. Agrupación de imágenes en una lista

Python

```
imagenes = [img1, img2, img3]
```

Se crea una lista que contiene las tres imágenes. Esto permite iterar o aplicar operaciones a todas ellas de manera más eficiente, facilitando la comparación entre las imágenes.

2.4. Detección automática de la imagen más grande

Python

```
mas_grande = max(imagenes, key=lambda img: img.shape[0] * img.shape[1])
```

Esta línea identifica cuál de las tres imágenes tiene mayor tamaño en términos de cantidad total de píxeles. La función `max()` recorre la lista de imágenes y para cada una calcula `img.shape[0] * img.shape[1]`, donde:

- `img.shape[0]` devuelve el alto de la imagen (número de filas)
- `img.shape[1]` devuelve el ancho de la imagen (número de columnas)

El producto de alto por ancho da el número total de píxeles de la imagen. La función `max()` selecciona la imagen con el valor más alto. La expresión `lambda` es una función anónima que se aplica a cada elemento de la lista.

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 3

2.5. Obtención de las dimensiones de la imagen más grande

Python

```
ancho = mas_grande.shape[1]
alto = mas_grande.shape[0]
```

Una vez identificada la imagen más grande, se extraen sus dimensiones:

- shape[1] obtiene el ancho (número de columnas)
- shape[0] obtiene el alto (número de filas)

Estas dimensiones serán el tamaño objetivo al que se redimensionarán todas las imágenes.

2.6. Visualización de dimensiones originales

Python

```
print("Dimensiones originales:")
print("Aguila:", img1.shape)
print("Gato:", img2.shape)
print("Lobo:", img3.shape)
print("")
print("Tamaño de la imagen mas grande:", alto, "x", ancho)
```

Las líneas muestran en la consola las dimensiones originales de cada imagen antes de ser modificadas. El atributo .shape devuelve una tupla con tres valores: (alto, ancho, número de canales de color). Para imágenes a color, el número de canales es 3 (RGB - Rojo, Verde, Azul). También se imprime el tamaño de la imagen más grande que servirá como referencia.

2.7. Redimensionamiento de las imágenes

Python

```
img1_redim = cv2.resize(img1, (ancho, alto))
img2_redim = cv2.resize(img2, (ancho, alto))
img3_redim = cv2.resize(img3, (ancho, alto))
```

La función cv2.resize() cambia las dimensiones de una imagen. Recibe dos parámetros principales:

- La imagen original que se quiere redimensionar
- Una tupla (ancho, alto) con las nuevas dimensiones

En este caso, todas las imágenes se redimensionan para que tengan exactamente el mismo ancho y alto que la imagen más grande. El resultado se almacena en nuevas variables (img1_redim, img2_redim, img3_redim) para preservar las originales.

2.8. Guardado de las imágenes redimensionadas

Python

```
cv2.imwrite("Imagenes/Aguila_redimensionada.jpg", img1_redim)
cv2.imwrite("Imagenes/Gato_redimensionado.jpg", img2_redim)
cv2.imwrite("Imagenes/Lobo_redimensionado.jpg", img3_redim)
print("")
print("Imagenes guardadas exitosamente")
```

La función cv2.imwrite() guarda una imagen en el disco. Recibe dos parámetros:

- La ruta y nombre del archivo de destino
- La imagen (arreglo) que se desea guardar

Las imágenes se guardan en la misma carpeta "Imágenes" con nombres que indican que fueron redimensionadas. Esto permite conservar las imágenes originales y tener las versiones modificadas para su uso en ejercicios posteriores.

2.9. Resultado

Después del proceso de redimensionamiento, la imagen más grande resultó ser Lobo.jpg con dimensiones de 500x755 píxeles, por lo que el programa tomó este tamaño como referencia y redimensionó Aguila.jpg (originalmente 400x600) y Gato.jpg (originalmente 300x400) a las mismas dimensiones de 500x755, logrando que las tres imágenes redimensionadas tengan exactamente el mismo tamaño.

	Aguila.jpg	Tipo: Archivo JPG Dimensiones: 400 x 600
	Aguila_redimensionada.jpg	Tipo: Archivo JPG Dimensiones: 500 x 755
	Gato.jpg	Tipo: Archivo JPG Dimensiones: 300 x 400
	Gato_redimensionado.jpg	Tipo: Archivo JPG Dimensiones: 500 x 755
	Lobo.jpg	Tipo: Archivo JPG Dimensiones: 500 x 755
	Lobo_redimensionado.jpg	Tipo: Archivo JPG Dimensiones: 500 x 755

3. Crear nueva imagen con canales de color combinados

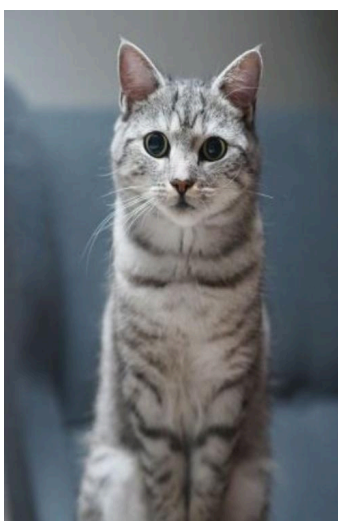
Se creo una nueva imagen combinando un canal de color de cada una de las tres imagenes originales. Se extrajo el canal rojo de Aguila.jpg, el canal verde de Gato.jpg y el canal azul de Lobo.jpg, luego se fusionaron en una sola imagen.

3.1. Carga de imágenes originales

Python

```
img1 = cv2.imread("Imagenes/Aguila.jpg")  
img2 = cv2.imread("Imagenes/Gato.jpg")  
img3 = cv2.imread("Imagenes/Lobo.jpg")
```

Se cargan tres imagenes desde la carpeta "Imagenes" utilizando la funcion cv2.imread(). Cada imagen se almacena como un arreglo tridimensional donde las dimensiones representan alto, ancho y canales de color (BGR). Las imágenes seleccionadas son un águila, un gato y un lobo, todas a color.



3.2. Unificación de dimensiones

Python

```
mas_grande = max([img1, img2, img3], key=lambda img: img.shape[0] * img.shape[1])  
ancho = mas_grande.shape[1]  
alto = mas_grande.shape[0]  
  
img1 = cv2.resize(img1, (ancho, alto))  
img2 = cv2.resize(img2, (ancho, alto))  
img3 = cv2.resize(img3, (ancho, alto))
```

Para poder combinar canales de color, las tres imágenes deben tener exactamente las mismas dimensiones. Por ello, se identifica automáticamente la imagen con mayor cantidad de píxeles (mayor área) calculando el producto de su alto por su ancho. Una vez identificada, se obtienen sus dimensiones (ancho y alto) y se redimensionan las tres imágenes a ese tamaño usando `cv2.resize()`. Este paso asegura que todas las imágenes tengan dimensiones compatibles.

3.3. Extracción de canales de color individuales

Python

```
canal_rojo = img1[:, :, 2] # Rojo de Aguila
canal_verde = img2[:, :, 1] # Verde de Gato
canal_azul = img3[:, :, 0] # Azul de Lobo
```

OpenCV almacena las imágenes en formato BGR (Azul, Verde, Rojo) en lugar de RGB. Por lo tanto:

- El canal Rojo se encuentra en el índice 2 de la tercera dimensión
- El canal Verde se encuentra en el índice 1
- El canal Azul se encuentra en el índice 0

Se extrae el canal Rojo de la imagen del águila, el canal Verde de la imagen del gato y el canal Azul de la imagen del lobo. La sintaxis `[:, :, n]` selecciona todos los píxeles (alto y ancho) pero solo el canal `n`. El resultado de cada extracción es una matriz bidimensional (escala de grises) que representa la intensidad de ese canal específico.

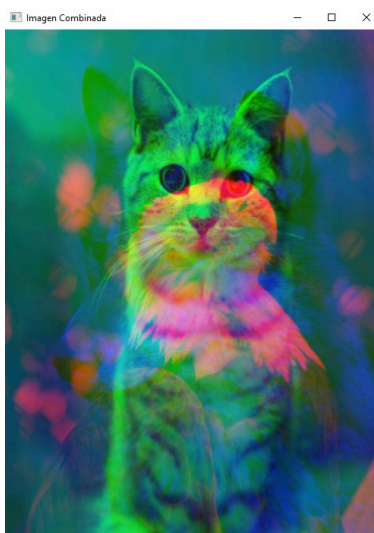
3.4. Combinación de canales

Python

```
imagen_combinada = cv2.merge([canal_azul, canal_verde, canal_rojo])
```

La función `cv2.merge()` toma una lista de canales individuales y los combina en una sola imagen a color. Es importante notar que el orden de los canales debe ser [Azul, Verde, Rojo] para mantener el formato BGR que OpenCV utiliza internamente. Como se extrajo el canal Rojo del águila, el canal Verde del gato y el canal Azul del lobo, la imagen resultante es una composición híbrida donde cada canal de color proviene de un animal diferente.

3.5. Resultado



Se obtuvo una imagen con colores completamente diferentes a las originales debido a que cada canal de color proviene de una imagen distinta. La imagen combinada fue guardada como combinada.jpg.

4. Conversión a negativo y escala de grises

Usando la imagen combinada del ejercicio anterior, se realizaron dos operaciones. Primero se convirtió cada píxel a su negativo invirtiendo los valores de color, luego se abrió esa imagen negativa en escala de grises y se guardó.

4.1. Conversión a negativo

Python

```
negativo = 255 - imagen_combinada
```

En una imagen digital, los valores de los píxeles en formato BGR (Azul, Verde, Rojo) típicamente van de 0 a 255, donde 0 representa el color más oscuro (negro) y 255 el más brillante (blanco puro para cada canal). Para obtener el negativo de una imagen, se invierte cada valor de píxel restando su valor actual de 255. Esto significa que:

- Un píxel con valor 0 (negro) se convierte en 255 (blanco)
- Un píxel con valor 255 (blanco) se convierte en 0 (negro)
- Un píxel con valor 128 (gris medio) se convierte en 127 (gris medio invertido)

La operación se aplica a todos los canales de color simultáneamente gracias a la vectorización de NumPy (OpenCV maneja las imágenes como arreglos de NumPy). El

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 8

resultado es una imagen que visualmente parece una "fotografía revelada" o un negativo fotográfico tradicional.

4.2. Guardado y visualización del negativo

Python

```
cv2.imwrite("Imagenes/negativo.jpg", negativo)
print("Imagen negativo guardada correctamente")
cv2.imshow("Negativo", negativo)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

La imagen resultante del negativo se guarda en el disco con el nombre "negativo.jpg" dentro de la carpeta "Imagenes". Luego se muestra en una ventana titulada "Negativo" utilizando cv2.imshow(). La función cv2.waitKey(0) mantiene la ventana abierta hasta que el usuario presiona una tecla, y cv2.destroyAllWindows() cierra la ventana para continuar con la ejecución del programa.

4.3. Conversión a escala de grises

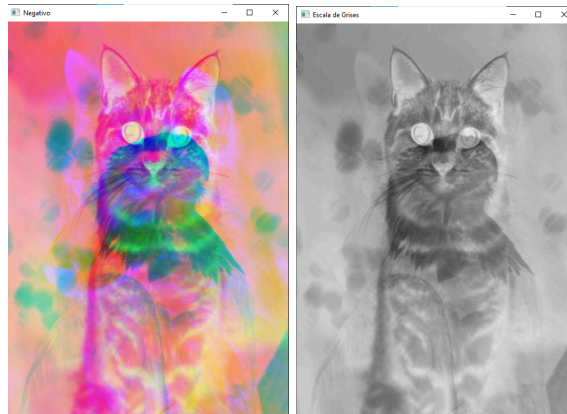
Python

```
gris = cv2.imread("Imagenes/negativo.jpg", cv2.IMREAD_GRAYSCALE)
```

Para convertir la imagen a escala de grises, se utiliza nuevamente cv2.imread() pero esta vez con un segundo parámetro: cv2.IMREAD_GRAYSCALE. Este parámetro indica a OpenCV que cargue la imagen directamente en escala de grises, es decir, como una matriz bidimensional donde cada pixel tiene un único valor de intensidad que va de 0 (negro) a 255 (blanco). OpenCV aplica internamente una fórmula de conversión que combina los tres canales de color en un solo valor de luminancia, típicamente utilizando la fórmula: $\text{Gris} = 0.299 \cdot \text{Rojo} + 0.587 \cdot \text{Verde} + 0.114 \cdot \text{Azul}$, que aproxima la percepción humana del brillo.

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 9

4.4. Resultados



Se obtuvieron dos imágenes. La primera es el negativo de la imagen combinada mostrando colores invertidos. La segunda es esa misma imagen negativa convertida a escala de grises. Ambas fueron guardadas correctamente como negativo.jpg y gris.jpg.

5. Aplicacion interactiva de visualizacion de canales de color

Se creo una aplicacion interactiva que permite al usuario alternar la visibilidad de los canales de color de una imagen en tiempo real usando el teclado.

5.1. Importación de librerías y carga de imagen

```
Python
import cv2
import numpy as np

imagen = cv2.imread("Imagenes/combinada.jpg")
```

Se importan dos librerías: OpenCV (cv2) para el procesamiento de imágenes y captura de teclas, y NumPy (np) que trabaja internamente con OpenCV para manejar los arreglos multidimensionales de las imágenes. Luego se carga la imagen "combinada.jpg" generada en el ejercicio anterior, la cual es una composicion hibrida donde cada canal de color proviene de un animal diferente.

5.2. Estado inicial de los canales

```
Python
canales = [True, True, True] # [Azul, Verde, Rojo]
```

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 10

Se crea una lista llamada canales que almacena el estado de visibilidad de cada canal de color. La lista sigue el orden [Azul, Verde, Rojo] correspondiente al formato BGR que utiliza OpenCV. Cada elemento es un valor booleano:

- True indica que el canal es visible
- False indica que el canal está oculto (sus valores se establecen a 0)

Inicialmente, los tres canales están activados (True), lo que significa que se muestra la imagen completa a color.

5.3. Instrucciones para el usuario

Python

```
print("Controles:")
print("Tecla 'r' - alternar canal Rojo")
print("Tecla 'g' - alternar canal Verde")
print("Tecla 'b' - alternar canal Azul")
print("Tecla 'q' - salir")
```

Se muestran en la consola las instrucciones de uso del programa. El usuario puede presionar las teclas 'r', 'g' o 'b' para alternar la visibilidad de los canales Rojo, Verde o Azul respectivamente. La tecla 'q' permite salir del programa de forma controlada.

5.4. Bucle principal de la aplicación

Python

```
while True:
    resultado = imagen.copy()
```

Se crea un bucle infinito (while True) que mantiene la aplicación ejecutándose hasta que el usuario decida salir. Dentro del bucle, primero se crea una copia de la imagen original utilizando imagen.copy(). Es importante trabajar con una copia porque se van a modificar los valores de los canales; de esta forma, la imagen original permanece intacta para cada iteración.

5.5. Ocultamiento de canales según el estado

Python

```
if not canales[0]: # Azul
    resultado[:, :, 0] = 0
if not canales[1]: # Verde
    resultado[:, :, 1] = 0
if not canales[2]: # Rojo
    resultado[:, :, 2] = 0
```

Se verifica el estado de cada canal. Si un canal tiene el valor False (oculto), se establecen todos sus valores a 0 utilizando la notación `resultado[:, :, indice] = 0`, donde:

- `:` selecciona todas las filas (alto)
- `:` selecciona todas las columnas (ancho)
- `índice` selecciona el canal específico (0=Azul, 1=Verde, 2=Rojo)

Asignar 0 a un canal significa que ese color no contribuye a la imagen resultante. Por ejemplo, si se oculta el canal Rojo, la imagen mostrará solo tonos que combinan Azul y Verde.

5.6. Visualización y captura de teclas

Python

```
cv2.imshow("Canales de Color - r/g/b para alternar", resultado)
tecla = cv2.waitKey(1) & 0xFF
```

La función `cv2.imshow()` muestra la imagen modificada en una ventana con el título indicativo de los controles. Luego, `cv2.waitKey(1)` espera 1 milisegundo por una pulsación de tecla. El `& 0xFF` es una máscara de bits que asegura obtener correctamente el código de la tecla presionada, independientemente del sistema operativo. El valor 1 permite que el bucle se actualice rápidamente, haciendo que la interfaz sea responsiva.

5.7. Procesamiento de las teclas presionadas

Python

```
if tecla == ord('r'):
    canales[2] = not canales[2]
    print("Canal Rojo:", "visible" if canales[2] else "oculto")
elif tecla == ord('g'):
    canales[1] = not canales[1]
    print("Canal Verde:", "visible" if canales[1] else "oculto")
elif tecla == ord('b'):
    canales[0] = not canales[0]
    print("Canal Azul:", "visible" if canales[0] else "oculto")
elif tecla == ord('q'):
    break
```

La función `ord()` convierte el carácter de la tecla presionada a su código ASCII para compararlo con las teclas esperadas ('r', 'g', 'b', 'q'). Cuando el usuario presiona:

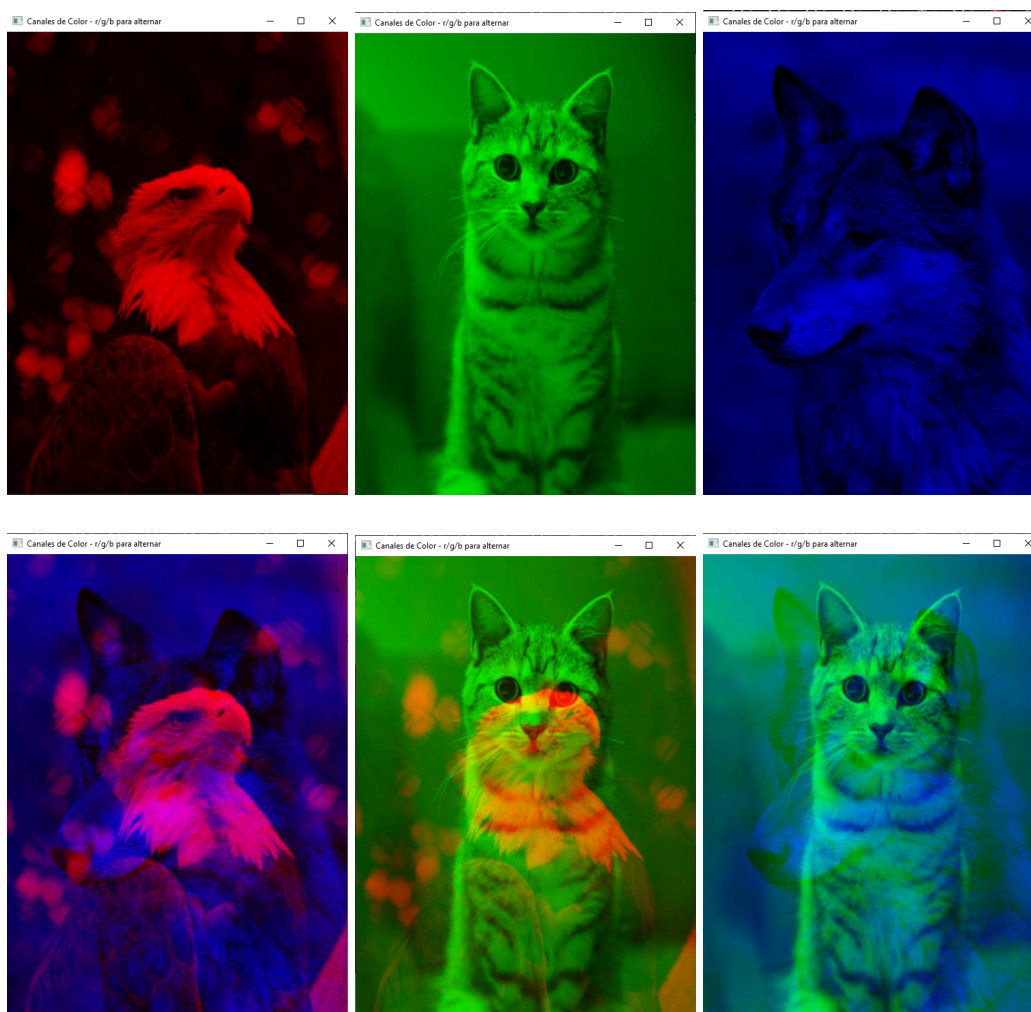
- 'r': Alterna el estado del canal Rojo (índice 2) usando `not` para invertir el valor booleano
- 'g': Alterna el estado del canal Verde (índice 1)

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 12

- 'b': Alterna el estado del canal Azul (índice 0)
- 'q': Sale del bucle con break

Cada vez que se alterna un canal, se imprime en consola el nuevo estado (visible u oculto) para informar al usuario.

5.8. Resultados



La aplicación muestra correctamente cada canal de forma individual y permite combinaciones entre ellos. Al desactivar dos canales se visualiza únicamente el canal restante en su color correspondiente (rojo, verde o azul).

6. Dibujo de figuras y texto en imágenes

Se abrió la imagen del gato, se dibujó un círculo sobre su cara y se agregó un texto descriptivo indicando que animal es. Finalmente se guardó la imagen con las anotaciones.

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 13

6.1. Dibujo del círculo sobre la cara

Python

```
cv2.circle(imagen, (150, 120), 80, (0, 255, 0), 3)
```

La función `cv2.circle()` dibuja un círculo sobre la imagen. Los parámetros son:

- `imagen`: La imagen sobre la cual se dibuja
- `(150, 120)`: Coordenadas del centro del círculo (x, y) en píxeles. Estos valores deben ajustarse según la posición de la cara del gato en la imagen específica
- `80`: Radio del círculo en píxeles
- `(0, 255, 0)`: Color del círculo en formato BGR. En este caso, (0, 255, 0) representa color verde puro (0 de Azul, 255 de Verde, 0 de Rojo)
- `3`: Grosor del borde del círculo en píxeles. Un valor positivo dibuja solo el contorno; un valor negativo rellenaría el círculo

El resultado es un círculo verde que rodea la cara del gato, destacando visualmente esa región.

6.2. Agregado de texto descriptivo

Python

```
cv2.putText(imagen, "Gato", (100, 230), cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 255, 0), 2)
```

La función `cv2.putText()` añade texto a la imagen. Los parámetros son:

- `imagen`: La imagen sobre la cual se escribe
- `"Gato"`: El texto que se va a mostrar
- `(100, 230)`: Coordenadas de la esquina inferior izquierda donde comienza el texto
- `cv2.FONT_HERSHEY_SIMPLEX`: Tipo de fuente utilizada. Es una fuente simple y legible
- `1`: Escala o tamaño de la fuente (1 es tamaño normal)
- `(0, 255, 0)`: Color del texto en formato BGR (verde)
- `2`: Grosor de las letras en píxeles

El texto "Gato" se coloca cerca del círculo para identificar claramente al animal en la imagen.

6.3. Resultados



Se obtuvo la imagen del gato con un círculo verde dibujado sobre su cara y el texto "Gato" agregado debajo del círculo. La imagen fue guardada correctamente como gato_etiquetado.jpg.

7. Aplicación de umbral binario

Se aplicó un umbral binario a la imagen del gato. Este proceso convierte cada pixel de la imagen en blanco o negro segun si su valor de intensidad supera o no el umbral establecido.

7.1. Carga de la imagen en escala de grises

```
Python  
imagen = cv2.imread("Imagenes/Gato.jpg", cv2.IMREAD_GRAYSCALE)
```

Se carga la imagen "Gato.jpg" utilizando el parámetro `cv2.IMREAD_GRAYSCALE`, lo que indica a OpenCV que convierta automáticamente la imagen a escala de grises durante la carga. El resultado es una matriz bidimensional donde cada pixel tiene un unico valor de intensidad que va de 0 (negro) a 255 (blanco). Es importante trabajar con una imagen en grises para aplicar el umbral binario, ya que este proceso opera sobre un solo canal de intensidad.

7.2. Aplicación del umbral binario

```
Python  
_, threshold = cv2.threshold(imagen, 127, 255, cv2.THRESH_BINARY)
```

La función `cv2.threshold()` aplica un umbral binario a la imagen. Los parámetros son:

- `imagen`: La imagen en escala de grises a procesar

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 15

- 127: Valor del umbral. Los píxeles con intensidad mayor a este valor serán tratados de una forma, y los menores de otra
- 255: Valor máximo que se asignará a los píxeles que superen el umbral (blanco)
- cv2.THRESH_BINARY: Tipo de umbral binario. Este modo funciona de la siguiente manera:
 - Si el valor del píxel es mayor al umbral (127), se asigna el valor máximo (255 = blanco)
 - Si el valor del píxel es menor o igual al umbral (127), se asigna 0 (negro)

La función retorna dos valores: el primero (que se asigna a `_`) es el valor del umbral utilizado, y el segundo (`threshold`) es la imagen resultante. Se usa `_` como nombre para indicar que ese valor no es necesario para el programa.

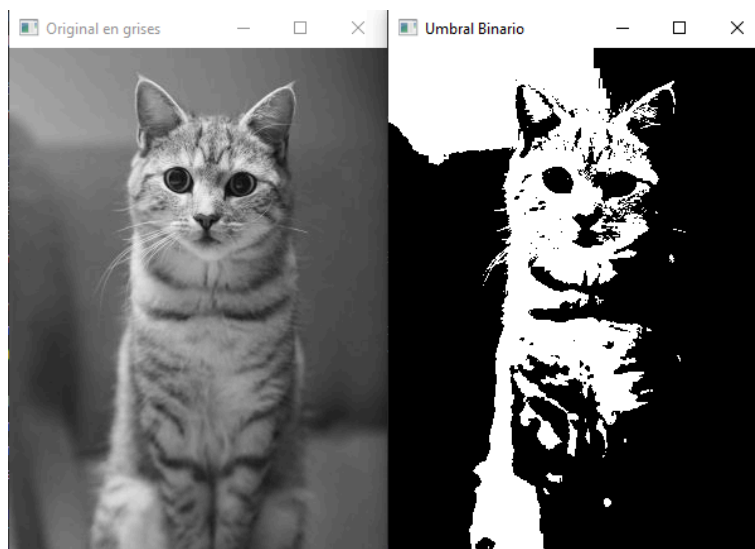
7.3. Guardado de la imagen resultante

Python

```
cv2.imwrite("Imagenes/threshold.jpg", threshold)
print("Imagen con umbral binario guardada correctamente")
```

La imagen resultante del umbral binario se guarda en el disco con el nombre "threshold.jpg" dentro de la carpeta "Imagenes". Esta imagen contiene únicamente píxeles de color blanco (255) y negro (0), sin tonos grises intermedios.

7.4. Resultados



Se obtuvieron dos ventanas para comparar. La imagen original en escala de grises y la imagen con el umbral binario aplicado donde se distinguen claramente dos rangos de

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 16

intensidad, los pixeles blancos representan las zonas mas claras y los negros las mas oscuras. La imagen fue guardada como threshold.jpg.

8. Programa de dibujo interactivo con eventos de mouse y teclado

8.1. Importación y creación del lienzo

Python

```
import cv2
import numpy as np

lienzo = np.ones((600, 800, 3), dtype=np.uint8) * 255
```

Se importan las librerías OpenCV y NumPy. Se crea un lienzo en blanco de 600 pixeles de alto por 800 pixeles de ancho con 3 canales de color (BGR). La función np.ones() crea una matriz de unos (valor 1) y se multiplica por 255 para obtener pixeles blancos. El tipo np.uint8 indica que los valores van de 0 a 255.

8.2. Inicialización de variables de control

Python

```
historial = []
figura_actual = "rectangulo"
punto_inicio = None
```

- historial: Lista vacía que almacenara copias del lienzo para permitir deshacer acciones
- figura_actual: Variable que define la figura a dibujar. Inicialmente se establece "rectangulo"
- punto_inicio: Variable que almacena las coordenadas donde el usuario presiona el mouse. Inicialmente es None

8.3. Instrucciones para el usuario

Python

```
print("Controles:")
print("Tecla 'r' - dibujar rectangulo")
print("Tecla 'c' - dibujar circulo")
print("Tecla 'l' - dibujar linea")
print("Tecla 'z' - deshacer")
print("Tecla 's' - guardar")
print("Tecla 'q' - salir")
```

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 17

Se muestran en consola todos los comandos disponibles para que el usuario sepa como interactuar con el programa.

8.4. Función de eventos del mouse

Python

```
def eventos_mouse(evento, x, y, flags, param):
    global lienzo, punto_inicio, historial
```

Se define una función que maneja los eventos del mouse. La instrucción global permite modificar las variables globales dentro de la función. Los parámetros son:

- evento: Tipo de evento (click, movimiento, etc.)
- x, y: Coordenadas donde ocurrió el evento
- flags: Estado de teclas especiales (no se usa)
- param: Parametros adicionales (no se usa)

8.5. Evento de presionar el mouse

Python

```
if evento == cv2.EVENT_LBUTTONDOWN:
    punto_inicio = (x, y)
    historial.append(lienzo.copy())
```

Cuando el usuario presiona el botón izquierdo del mouse, se guarda la posición inicial en punto_inicio. Además, se guarda una copia del estado actual del lienzo en el historial usando lienzo.copy(). Esto permite deshacer la acción posteriormente. Guardar antes de dibujar es importante porque almacena el estado previo a la nueva figura.

8.6. Evento de soltar el mouse (dibujo de figuras)

Python

```
elif evento == cv2.EVENT_LBUTTONUP:
    if punto_inicio is not None:
        if figura_actual == "rectangulo":
            cv2.rectangle(lienzo, punto_inicio, (x, y), (0, 0, 255), 2)
        elif figura_actual == "circulo":
            radio = int(((x - punto_inicio[0])**2 + (y - punto_inicio[1])**2)**0.5)
            cv2.circle(lienzo, punto_inicio, radio, (0, 255, 0), 2)
        elif figura_actual == "linea":
            cv2.line(lienzo, punto_inicio, (x, y), (255, 0, 0), 2)
    punto_inicio = None
```

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 18

Cuando el usuario suelta el botón del mouse y existe un punto de inicio, se dibuja la figura según la seleccion actual:

- Rectángulo: Se dibuja usando `cv2.rectangle()` desde el punto de inicio hasta el punto final. El color es rojo (0,0,255) en BGR
- Círculo: Se calcula el radio como la distancia euclidiana entre el punto de inicio y el punto final usando la fórmula $\sqrt{(x_2-x_1)^2 + (y_2-y_1)^2}$. El círculo se dibuja con centro en `punto_inicio` y color verde (0,255,0)
- Línea: Se dibuja una línea recta entre el punto de inicio y el punto final con color azul (255,0,0)

Finalmente, `punto_inicio` se reinicia a `None`.

8.7. Configuración de la ventana y el callback

Python

```
cv2.namedWindow("Dibujo")
cv2.setMouseCallback("Dibujo", eventos_mouse)
```

`cv2.namedWindow()` crea una ventana con el nombre "Dibujo". `cv2.setMouseCallback()` asigna la función `eventos_mouse` a esa ventana, lo que significa que cada vez que ocurra un evento del mouse dentro de la ventana, se llamará automáticamente a esa función.

8.8. Bucle principal de la aplicación

Python

```
while True:
    cv2.imshow("Dibujo", lienzo)
    tecla = cv2.waitKey(1) & 0xFF
```

El bucle infinito mantiene el programa ejecutándose. En cada iteración se muestra el lienzo actualizado y se captura la tecla presionada por el usuario. El `& 0xFF` es una máscara que asegura obtener el código correcto de la tecla.

8.9. Procesamiento de teclas

Python

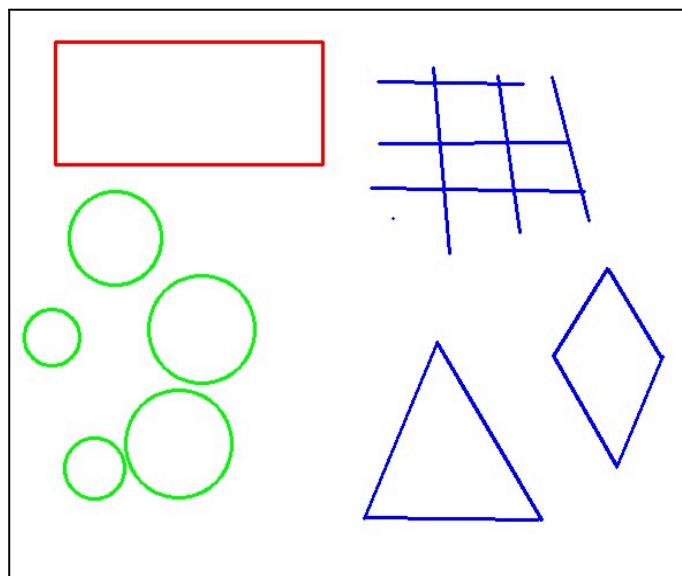
```
if tecla == ord('r'):
    figura_actual = "rectangulo"
    print("Figura: rectangulo")
elif tecla == ord('c'):
    figura_actual = "circulo"
    print("Figura: circulo")
```

```
elif tecla == ord('l'):  
    figura_actual = "linea"  
    print("Figura: linea")  
elif tecla == ord('z'):  
    if historial:  
        lienzo = historial.pop()  
        print("Deshacer realizado")  
    else:  
        print("No hay mas acciones para deshacer")  
elif tecla == ord('s'):  
    cv2.imwrite("Imagenes/dibujo.jpg", lienzo)  
    print("Dibujo guardado como dibujo.jpg")  
elif tecla == ord('q'):  
    break
```

Según la tecla presionada:

- 'r': Cambia la figura actual a rectángulo
- 'c': Cambia la figura actual a círculo
- 'l': Cambia la figura actual a línea
- 'z': Deshace la última acción. Si hay elementos en el historial, se recupera el último estado guardado usando `historial.pop()` que elimina y devuelve el último elemento
- 's': Guarda el dibujo actual en la carpeta "Imagenes" con nombre "dibujo.jpg"
- 'q': Sale del bucle y termina el programa

8.10. Resultados



	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 20

Se obtuvo un programa funcional donde el usuario puede dibujar rectángulos, círculos y líneas de diferentes colores, deshacer trazos y guardar el resultado final como dibujo.jpg.

II. SOLUCIÓN DEL CUESTIONARIO

1. ¿Qué otros modelos de colores existen para las imágenes?

Ademas del modelo RGB existen otros modelos:

- HSV (Hue, Saturation, Value) - Representa el color por su tono, saturación y brillo. Es muy utilizado para detección de colores específicos en imágenes.
- HSL (Hue, Saturation, Lightness) - Similar al HSV pero usa luminosidad en lugar de brillo.
- CMYK (Cyan, Magenta, Yellow, Black) - Modelo utilizado en impresión. Los colores se forman restando luz en lugar de sumandola.
- LAB (Lab)* - Modelo basado en la percepción humana del color. Es independiente del dispositivo y útil para comparar colores.

2. ¿Qué otros cambios se pueden realizar a las imágenes con OpenCV?

OpenCV ofrece una variedad de operaciones sobre imágenes, entre las más importantes:

- Rotación y traslación - Girar o mover una imagen usando transformaciones afines.
- Detección de bordes - Usando algoritmos como Canny o Sobel para identificar contornos.
- Suavizado y desenfoque - Aplicando filtros como GaussianBlur o MedianBlur para reducir ruido.
- Detección de rostros - Usando clasificadores en cascada como Haar Cascade.
- Morfología - Operaciones como erosión y dilatación para modificar formas en imágenes binarias.
- Histogramas - Analizar y ecualizar la distribución de intensidades de una imagen.
- Detección de contornos - Usando cv2.findContours para identificar formas en una imagen.
- Transformaciones de perspectiva - Corregir la perspectiva de una imagen.

3. ¿Cuál sería el uso del binary threshold en el procesamiento de imágenes?

El umbral binario es una técnica fundamental en el procesamiento de imágenes con los siguientes usos principales:

- Segmentación de objetos - Permite separar un objeto del fondo de una imagen convirtiendo los pixeles en blanco o negro segun su intensidad.
- Detección de formas - Al simplificar la imagen a dos valores facilita la detección de contornos y formas geométricas.
- OCR (Reconocimiento de texto) - Se aplica antes de reconocer texto en imágenes para separar las letras del fondo.
- Visión artificial - En sistemas de detección de objetos el threshold permite aislar regiones de interés de forma rápida y eficiente.

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 21

- Reducción de ruido - Elimina variaciones menores de intensidad que no son relevantes para el análisis.

III. CONCLUSIONES

- El procesamiento de imágenes digitales mediante OpenCV permite manipular los canales de color de forma individual, lo que posibilita crear nuevas imágenes combinando características de diferentes fuentes y obtener efectos visuales que no son posibles con las imágenes originales.
- La conversión de imágenes a negativo y escala de grises son operaciones fundamentales en el procesamiento de imágenes. El negativo se obtiene de forma simple restando cada valor de pixel a 255, mientras que la escala de grises reduce la información de color a un único canal de intensidad facilitando análisis posteriores.
- El umbral binario es una técnica esencial en visión artificial ya que permite segmentar una imagen en dos regiones claramente diferenciadas, simplificando el análisis y la detección de objetos al eliminar información irrelevante y resaltar las características de interés.
- La implementación de aplicaciones interactivas con OpenCV mediante eventos de mouse y teclado demuestra que la librería no solo sirve para procesamiento estático de imágenes sino también para desarrollar herramientas dinámicas donde el usuario puede interactuar en tiempo real con las imágenes.
- El trabajo con arreglos NumPy como estructura base de las imágenes en OpenCV permite comprender que una imagen digital no es más que una matriz de valores numéricos, lo que facilita la aplicación de operaciones matemáticas directamente sobre los píxeles para lograr transformaciones precisas y eficientes.

RETROALIMENTACIÓN GENERAL

REFERENCIAS Y BIBLIOGRAFÍA

- [1] OpenCV, "OpenCV: Image Processing," OpenCV Documentation, 2024. [En línea]. Disponible en: https://docs.opencv.org/4.x/d2/d96/tutorial_py_table_of_contents_imgproc.html
- [2] NumPy, "NumPy: the absolute basics for beginners," NumPy Documentation, 2024. [En línea]. Disponible en: https://numpy.org/doc/stable/user/absolute_beginners.html
- [3] Real Python, "Image Processing With the Python Pillow Library," Real Python, 2024. [En línea]. Disponible en: <https://realpython.com/image-processing-with-the-python-pillow-library/>
- [4] OpenCV, "OpenCV: Changing Colorspaces," OpenCV Documentation, 2024. [En línea]. Disponible en: https://docs.opencv.org/4.x/df/d9d/tutorial_py_colorspaces.html

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 22

[5] OpenCV, "OpenCV: Image Thresholding," *OpenCV Documentation*, 2024. [En línea]. Disponible en: https://docs.opencv.org/4.x/d7/d4d/tutorial_py_thresholding.html

[6] OpenCV, "OpenCV: Mouse as a Paint-Brush," *OpenCV Documentation*, 2024. [En línea]. Disponible en: https://docs.opencv.org/4.x/db/d5b/tutorial_py_mouse_handling.html

[7] OpenCV, "OpenCV: Drawing Functions," *OpenCV Documentation*, 2024. [En línea]. Disponible en: https://docs.opencv.org/4.x/dc/da5/tutorial_py_drawing_functions.html

[8] OpenCV, "OpenCV: Geometric Transformations of Images," *OpenCV Documentation*, 2024. [En línea]. Disponible en: https://docs.opencv.org/4.x/da/d6e/tutorial_py_geometric_transformations.html